

Word2Vec and CNN Architectures

Assignment

1. Word2Vec Embeddings

Word2Vec is a neural network-based technique used to convert words into numerical vectors called word embeddings. These embeddings capture semantic meaning, so words with similar meanings have similar vector representations. Word2Vec was introduced by Google researchers in 2013 and is widely used in Natural Language Processing (NLP) tasks such as text classification, machine translation, and sentiment analysis.

Word2Vec uses a shallow neural network with one hidden layer and learns word relationships by predicting words based on their context. It has two main architectures: CBOW (Continuous Bag of Words) and Skip-gram.

```
!pip install gensim scikit-learn matplotlib

from gensim.models import Word2Vec

from nltk.tokenize import word_tokenize

import nltk

nltk.download('punkt')

nltk.download('punkt_tab') # Added to download the missing resource

# Sample corpus

corpus = [

    "Natural language processing is a fascinating field",

    "Word embeddings capture semantic meanings",

    "NLP is used in chatbots and virtual assistants",

    "Word2Vec is a powerful tool for creating word embeddings"

]
```

```
# Tokenize sentences

tokenized_corpus = [word_tokenize(sentence.lower()) for sentence in
corpus]

print(tokenized_corpus)

# Train Word2Vec model

model = Word2Vec(sentences=tokenized_corpus, vector_size=100, window=5,
min_count=1, workers=4)

# Save the model

model.save("word2vec.model")

model = Word2Vec.load("word2vec.model")

# Get vectors for a subset of words

words = list(model.wv.index_to_key)[:10] # Select the first 10 words

print(words)

word_vectors = [model.wv[word] for word in words]

print(word_vectors)

from sklearn.decomposition import PCA

# Apply PCA for dimensionality reduction

pca = PCA(n_components=2)

pca_result = pca.fit_transform(word_vectors)

import matplotlib.pyplot as plt

# Plot the words in 2D space

plt.figure(figsize=(10, 5))

plt.scatter(pca_result[:, 0], pca_result[:, 1])
```

```

# Annotate the points with the words

for i, word in enumerate(words):

    plt.annotate(word, xy=(pca_result[i, 0], pca_result[i, 1]))

plt.title("2D Visualization of Word Embeddings")

plt.xlabel("PCA Component 1")

plt.ylabel("PCA Component 2")

plt.grid(True)

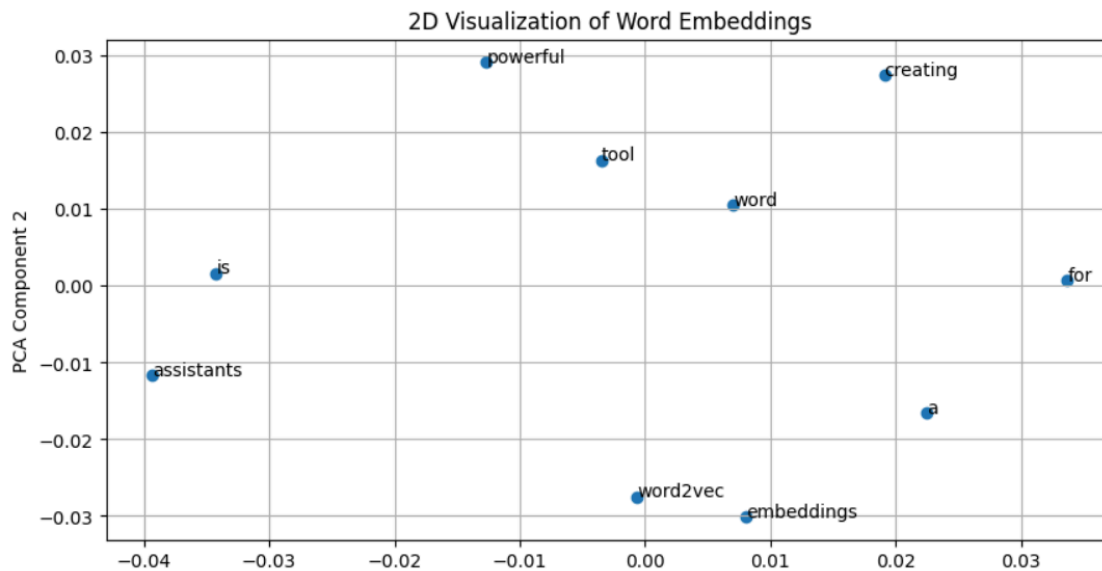
plt.show()

```

```

Successfully installed gensim-4.4.0
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
[['natural', 'language', 'processing', 'is', 'a', 'fascinating', 'field'], ['word', 'embeddings', 'capture', 'semantic', 'meanings'],
['is', 'embeddings', 'word', 'a', 'creating', 'for', 'tool', 'powerful', 'word2vec', 'assistants']]
[array([-5.3482049e-04,  2.3825991e-04,  5.1056668e-03,  9.0133334e-03,
        -9.3041826e-03, -7.1184216e-03,  6.4591165e-03,  8.9739896e-03,
        -5.0175558e-03, -3.7653942e-03,  7.3820921e-03, -1.5338163e-03,
        -4.5345500e-03,  6.5530594e-03, -4.8587834e-03, -1.8163866e-03,
        2.8779216e-03,  9.9028728e-04, -8.2870843e-03, -9.4514126e-03,
        7.3109688e-03,  5.0709019e-03,  6.7580235e-03,  7.6032913e-04,
        6.3481932e-03, -3.4047901e-03, -9.4892143e-04,  5.7721287e-03,
        -7.5215534e-03, -3.9352756e-03, -7.5127776e-03, -9.3112560e-04,
        9.5384931e-03, -7.3194429e-03, -2.3347498e-03, -1.9366210e-03,
        8.0779372e-03, -5.9318673e-03,  4.1685435e-05, -4.7525410e-03,
        -9.6020838e-03,  5.0074058e-03, -8.7600304e-03, -4.3903729e-03,
        -3.4098470e-05, -2.9744051e-04, -7.6616611e-03,  9.6138855e-03,
        4.9824370e-03,  9.2338668e-03, -8.1582749e-03,  4.4954196e-03,
        -4.1358816e-03,  8.2416082e-04,  8.4963180e-03, -4.4610659e-03,
        4.5164647e-03, -6.7853210e-03, -3.5473362e-03,  9.3990080e-03,
        -1.5771524e-03,  3.2147145e-04, -4.1403268e-03, -7.6813265e-03,
        -1.5084570e-03,  2.4713580e-03, -8.8901544e-04,  5.5368468e-03,
        -2.7449303e-03,  2.2591040e-03,  5.4555875e-03,  8.3476352e-03,
        -1.4524100e-03, -9.2081940e-03,  4.3700617e-03,  5.7310116e-04,
        7.4422611e-03, -8.1312557e-04, -2.6384799e-03, -8.7542878e-03,
        -8.5941557e-04,  2.8278236e-03,  5.4004234e-03,  7.0539773e-03,
        -5.7027326e-03,  1.8585746e-03,  6.0892231e-03, -4.7974382e-03,
        -3.1036362e-03,  6.7990194e-03,  1.6316617e-03,  1.8902181e-04,
        3.4705724e-03,  2.1915925e-04,  9.6204914e-03,  5.0593903e-03,
        -8.9182686e-03, -7.0416410e-03,  9.0250163e-04,  6.3918498e-03],

```



2. What is CBOW?

CBOW (Continuous Bag of Words) is one of the two Word2Vec architectures. In CBOW, the model predicts a target word based on its surrounding context words. For example, in the sentence 'The cat sits on the mat', if the target word is 'sits', the context words could be 'The', 'cat', 'on', 'the'.

CBOW works faster and is efficient for large datasets. It performs well with frequent words but may not represent rare words as effectively as Skip-gram.

```
# Install gensim

!pip install gensim

from gensim.models import Word2Vec

# Sample corpus

sentences = [

    ["i", "love", "Generative AI"],

    ["Generative AI", "is", "fun"],

    ["i", "love", "machine", "learning"]

]
```

```
# Train CBOW model (sg=0 means CBOW)

model = Word2Vec(sentences, vector_size=50, window=2, min_count=1, sg=0)

# Get word vector

print(model.wv["Generative AI"])

# Similar words

print(model.wv.most_similar("love"))
```

```
... Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.4.0)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.1)
[-0.01648536  0.01859871 -0.00039532 -0.00393455  0.00920726 -0.00819063
  0.00548623  0.01387993  0.01213085 -0.01502159  0.0187647  0.00934362
  0.00793224 -0.01248701  0.01691996 -0.00430033  0.01765038 -0.01072401
 -0.01625884  0.01364912  0.00334239 -0.00439702  0.0190272  0.01898771
 -0.01954809  0.00501046  0.01231338  0.00774491  0.00404557  0.000861
  0.00134726 -0.00764127 -0.0142805 -0.00417774  0.0078478  0.01763737
  0.0185183 -0.01195187 -0.01880534  0.01952875  0.00685957  0.01033223
  0.01256469 -0.00560853  0.01464541  0.00566054  0.00574201 -0.00476074
 -0.0062565 -0.00474028]
[(' Generative AI', 0.13204392790794373), ('learning', 0.1267007291316986), ('i', 0.042373016476631165), ('is', 0.012442179024219513), ('machine', -0.01447527389973402), ('
```

3. What is GloVe?

GloVe (Global Vectors for Word Representation) is another word embedding technique developed by Stanford University. Unlike Word2Vec, which is prediction-based, GloVe is count-based and uses a global word-word co-occurrence matrix to learn word vectors.

GloVe captures both local context (like Word2Vec) and global statistical information. It is trained by factorizing the co-occurrence matrix to obtain meaningful word embeddings.

```
!wget http://nlp.stanford.edu/data/glove.6B.zip
```

```
!unzip glove.6B.zip
```

```
# Load glove vectors
```

```
import numpy as np
```

```
embeddings = {}
```

```
with open("glove.6B.50d.txt", encoding="utf8") as f:
```

```

for line in f:

    values = line.split()

    word = values[0]

    vector = np.asarray(values[1:], dtype="float32")

    embeddings[word] = vector

print("Vector for 'king':\n", embeddings["king"])

```

```

Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:80... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://nlp.stanford.edu/data/glove.6B.zip [following]
--2026-02-16 06:24:01-- https://nlp.stanford.edu/data/glove.6B.zip
Connecting to nlp.stanford.edu (nlp.stanford.edu)|171.64.67.140|:443... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip [following]
--2026-02-16 06:24:01-- https://downloads.cs.stanford.edu/nlp/data/glove.6B.zip
Resolving downloads.cs.stanford.edu (downloads.cs.stanford.edu)... 171.64.64.22
Connecting to downloads.cs.stanford.edu (downloads.cs.stanford.edu)|171.64.64.22|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 862182613 (822M) [application/zip]
Saving to: 'glove.6B.zip'

glove.6B.zip      100%[=====>] 822.24M  5.01MB/s   in 2m 39s

2026-02-16 06:26:40 (5.18 MB/s) - 'glove.6B.zip' saved [862182613/862182613]

Archive: glove.6B.zip
  inflating: glove.6B.50d.txt
  inflating: glove.6B.100d.txt
  inflating: glove.6B.200d.txt
  inflating: glove.6B.300d.txt
Vector for 'king':
[ 0.50451  0.68607 -0.59517 -0.022801  0.60046 -0.13498 -0.08813
  0.47377 -0.61798 -0.31012 -0.076666  1.493   -0.034189 -0.98173
  0.68229  0.81722 -0.51874 -0.31503 -0.55809  0.66421  0.1961
 -0.13495 -0.11476 -0.30344  0.41177 -2.223   -1.0756  -1.0783
 -0.34354  0.33505  1.9927  -0.04234 -0.64319  0.71125  0.49159
  0.16754  0.34344 -0.25663 -0.8523   0.1661   0.40102  1.1685
 -1.0137  -0.21585 -0.15155  0.78321 -0.91241 -1.6106  -0.64426
 -0.51042 ]

```

4. Difference Between Word2Vec and GloVe

1. Word2Vec is a prediction-based model, while GloVe is a count-based model.
2. Word2Vec uses local context windows, whereas GloVe uses global word co-occurrence statistics.

3. Word2Vec trains a shallow neural network; GloVe factorizes a co-occurrence matrix.
4. GloVe captures global corpus statistics better, while Word2Vec is efficient and easier to train.

Feature	Word2Vec	GloVe
Model Type	Prediction-based model	Count-based model
Context Used	Uses local context windows	Uses global word co-occurrence statistics
Training Method	Trains a shallow neural network	Factorizes a co-occurrence matrix
Performance	Efficient and easier to train	Captures global corpus statistics better

5. What is LeNet?

LeNet is one of the earliest Convolutional Neural Network (CNN) architectures, developed by Yann LeCun in 1998. It was designed for handwritten digit recognition using the MNIST dataset.

LeNet consists of convolutional layers, pooling layers, and fully connected layers. It introduced the concept of using convolution and pooling for image feature extraction. LeNet laid the foundation for modern deep learning architectures in computer vision.

```
import tensorflow as tf

from tensorflow.keras import layers, models

# Load MNIST

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
```

```

x_train = x_train.reshape(-1,28,28,1)/255.0

x_test = x_test.reshape(-1,28,28,1)/255.0

# LeNet model

model = models.Sequential([

    layers.Conv2D(6, (5,5), activation='relu', input_shape=(28,28,1)),

    layers.AveragePooling2D((2, 2)),

    layers.Conv2D(16, (5,5), activation='relu'),

    layers.AveragePooling2D((2, 2)),

    layers.Flatten(),

    layers.Dense(120, activation='relu'),

    layers.Dense(84, activation='relu'),

    layers.Dense(10, activation='softmax')

])

model.compile(optimizer='adam',

              loss='sparse_categorical_crossentropy',

              metrics=['accuracy'])

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` arg
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/3
1875/1875 — 59s 29ms/step - accuracy: 0.8455 - loss: 0.4934
Epoch 2/3
1875/1875 — 35s 19ms/step - accuracy: 0.9720 - loss: 0.0923
Epoch 3/3
1875/1875 — 34s 18ms/step - accuracy: 0.9818 - loss: 0.0606
<keras.src.callbacks.history.History at 0x7b945194f440>

```

6. What is LlamaIndex?

LlamaIndex is a framework that connects Large Language Models (LLMs) to external data such as PDFs, documents, databases, and websites. It is mainly used in Retrieval-Augmented

Generation (RAG) systems. LlamaIndex converts data into embeddings and stores them in a vector database. When a user asks a question, it retrieves relevant information from the stored data. This retrieved context is then given to the LLM to generate accurate answers. It helps reduce hallucinations and improves response quality.

7. What is AlexNet?

AlexNet is a deep convolutional neural network developed by Alex Krizhevsky in 2012. It won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) in 2012 and significantly improved image classification accuracy.

AlexNet consists of 5 convolutional layers followed by 3 fully connected layers. It introduced important techniques such as ReLU activation, dropout to reduce overfitting, and GPU training for faster computation.

8. What is VGGNet?

VGGNet is a deep CNN architecture developed by the Visual Geometry Group (VGG) at Oxford University in 2014. It is known for its simplicity and uniform architecture.

VGGNet uses very small 3x3 convolution filters and increases depth significantly. Common versions include VGG16 and VGG19, with 16 and 19 layers respectively. Although VGGNet achieves high accuracy, it requires large memory and computational power.

```
import tensorflow as tf

from tensorflow.keras.applications import VGG16

from tensorflow.keras import layers, models

# Load pre-trained VGG16 (without top layer)

base_model = VGG16(weights='imagenet', include_top=False,
input_shape=(224,224,3))

model = models.Sequential([

    base_model,

    layers.Flatten(),
```

```

layers.Dense(256, activation='relu'),

layers.Dense(10, activation='softmax')

])

```

```
model.summary()
```

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5
58889256/58889256 — 0s 0us/step
Model: "sequential_1"

Layer (type)	Output Shape	Param #
vgg16 (Functional)	(None, 7, 7, 512)	14,714,688
flatten_1 (Flatten)	(None, 25088)	0
dense_3 (Dense)	(None, 256)	6,422,784
dense_4 (Dense)	(None, 10)	2,570

Total params: 21,140,042 (80.64 MB)
Trainable params: 21,140,042 (80.64 MB)
Non-trainable params: 0 (0.00 B)

9. What is Skip-gram?

Skip-gram is the second architecture of Word2Vec. Unlike CBOW, Skip-gram predicts the surrounding context words given a target word. For example, if the input word is 'sits', the model tries to predict words like 'The', 'cat', 'on', 'the'.

Skip-gram performs better for smaller datasets and rare words. It captures semantic relationships effectively but is computationally slower than CBOW.

```

!pip install gensim

from gensim.models import Word2Vec

sentences = [

    ["i", "love", "Generative AI"],

    ["Generative AI", "is", "fun"],

    ["deep", "learning", "is", "powerful"]

]

# sg=1 means Skip-Gram

```

```
model = Word2Vec(sentences, vector_size=50, window=2, min_count=1, sg=1)
```

```
print(model.wv["learning"])
```

```
print(model.wv.most_similar("Generative AI"))
```

```
Requirement already satisfied: gensim in /usr/local/lib/python3.12/dist-packages (4.4.0)
Requirement already satisfied: numpy>=1.18.5 in /usr/local/lib/python3.12/dist-packages (from gensim) (2.0.2)
Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from gensim) (1.16.3)
Requirement already satisfied: smart_open>=1.8.1 in /usr/local/lib/python3.12/dist-packages (from gensim) (7.5.0)
Requirement already satisfied: wrapt in /usr/local/lib/python3.12/dist-packages (from smart_open>=1.8.1->gensim) (2.1.1)
[ 1.56351421e-02 -1.90203730e-02 -4.11062239e-04  6.93839323e-03
 -1.87794445e-03  1.67635437e-02  1.80215668e-02  1.30730132e-02
 -1.42324204e-03  1.54208085e-02 -1.70686692e-02  6.41421322e-03
 -9.27599426e-03 -1.01779103e-02  7.17923651e-03  1.07406788e-02
 1.55390287e-02 -1.15330126e-02  1.48667218e-02  1.32509926e-02
 -7.41960062e-03 -1.74912829e-02  1.08749345e-02  1.30195115e-02
 -1.57510047e-03 -1.34197120e-02 -1.41718509e-02 -4.99412045e-03
 1.02865072e-02 -7.33047491e-03 -1.87401194e-02  7.65347946e-03
 9.76895820e-03 -1.28571270e-02  2.41711619e-03 -4.14975407e-03
 4.88066689e-05 -1.97670180e-02  5.38400887e-03 -9.50021297e-03
 2.17529293e-03 -3.15244915e-03  4.39334614e-03 -1.57631524e-02
 -5.43436781e-03  5.32639725e-03  1.06933638e-02 -4.78302967e-03
 -1.90201886e-02  9.01175756e-03]
[('deep', 0.12486250698566437), ('love', 0.07399576157331467), ('is', 0.04237300902605057), ('fun', 0.018277151510119438), ('powerful', 0.011071980930864811), ('
```